



Agent Evaluation and Quality Assurance

Welcome to Module 4: Agent Evaluation and Quality Assurance.

So far in the course we've discussed deployment, CI/CD, and observability for agents.

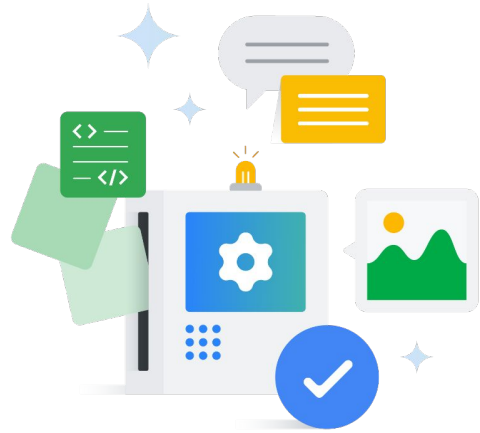
Now we will discuss agent evaluation.

Unlike traditional software testing, evaluating AI agents involves measuring not just outputs, but also reasoning steps, tool usage, and overall task completion.

This module focuses on how to systematically test, evaluate, and improve AI agents before and after deployment.

In this module, you learn to ...

- 01 Perform validation on model responses
- 02 Evaluate agent behavior, tool usage, and trajectory correctness
- 03 Create and manage evalsets using ADK Web UI
- 04 Evaluate evalsets with ADK UI, CLI, or code
- 05 Use the ADK Evaluation Tool



In this module, we'll cover five key capabilities.

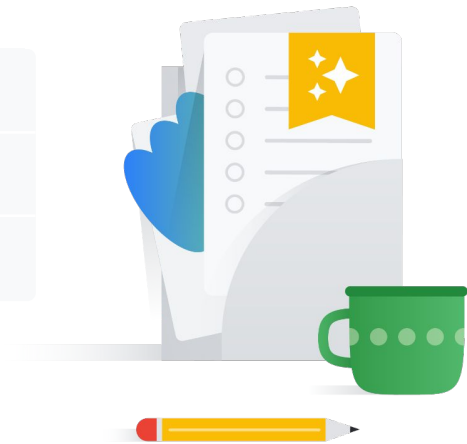
1. How to validate model responses.
2. Evaluate agent behavior, including tool usage and reasoning trajectories.
3. introduce the Agent Platform Generative AI Evaluation Service, which helps evaluate model responses at scale.
4. Create and manage evaluation datasets, also called evalsets, using the ADK web interface.
5. Run evaluations using the ADK Web UI, CLI, or code-based testing.

Topics

01 Testing generative AI model responses

02 Evaluating model responses

03 Lab: Evaluating Agents with ADK



The module is divided into three main sections.

- First, we'll discuss testing generative AI model responses.
- Second, we'll explore agent evaluation techniques and metrics.
- Finally, we'll complete a lab Evaluating Agents with ADK.

Functions that return results from LLMs need to be tested like any other function

- Functions that return deterministic results are simple
 - Using an LLM for classification or sentiment analysis are examples
 - Tested like any other function
 - These tasks are easier to evaluate because they have well-defined outputs, but models may still produce incorrect results
- Model responses that are indeterminate are more difficult to test
 - There is no single right answer to: “Write me a response to this email”
 - To automate testing use an LLM to evaluate correctness
- LLM-based evaluation is approximate and should be validated or calibrated against human judgment when possible

When LLMs are used inside applications, they are often embedded within functions.

These functions should be tested just like any other part of your application.

For deterministic tasks—such as classification or sentiment analysis—testing is straightforward. The model should produce a specific answer.

However, open-ended generation tasks are much harder to test because there may be many acceptable responses.

One common technique is to use another LLM to evaluate the output, sometimes called LLM-as-a-judge.

When working with functions that rely on large language models, or LLMs, it's essential to treat testing as a core part of your development process. Like any function in your application, functions that return results from LLMs must be rigorously tested—but the approach can vary depending on the nature of the output.

LLM classification example

```
def getPositiveOrNegative(prompt):  
    response = model.generate_content(  
        """Context: You look at messages and categorize them as  
        positive, negative, or neutral.  
  
        Output only Positive, Negative, or Neutral.  
  
        Message: {0}.  
        Output: """.format(prompt)  
    )  
    return response.text.strip()  
  
print(getPositiveOrNegative("Dinner was cold and undercooked."))
```

➡ Negative

This slide shows an example of using a model for sentiment classification.

The model reads a message and returns one of three labels: positive, negative, or neutral.

Because the expected output is structured and constrained, this type of task is relatively easy to test.

Unit testing a deterministic function

```
import unittest

class TestPositiveOrNegative(unittest.TestCase):
    def test_getPositiveOrNegative1(self):
        response = getPositiveOrNegative("Dinner was Great")
        self.assertEqual(response, "Positive")

    def test_getPositiveOrNegative2(self):
        response = getPositiveOrNegative("That broccoli was undercooked and cold")
        self.assertEqual(response, "Negative")

    def test_getPositiveOrNegative3(self):
        response = getPositiveOrNegative("We went to the new italian place for dinner")
        self.assertEqual(response, "Neutral")

unittest.main(argv=[''], verbosity=2)
```

```
test_getPositiveOrNegative1 (__main__.TestPositiveOrNegative) ... ok
test_getPositiveOrNegative2 (__main__.TestPositiveOrNegative) ... ok
test_getPositiveOrNegative3 (__main__.TestPositiveOrNegative) ... ok

-----
Ran 3 tests in 0.768s

OK
```

Google Cloud

6

Here we see how standard Python unit testing frameworks can validate model behavior.

Each test sends a prompt to the classification function and checks whether the returned label matches the expected value.

This approach works well when the output space is clearly defined.

LLM text generation example

```
def writePost(prompt):  
    response = model.generate_content(  
        """Context: You write posts for the Marketing Department at Luigi's Italian Cafe.  
  
        1. Keep your posts below 100 characters  
        2. Include the hashtag #EatAtLuigis at the end of every post  
  
        Input: Write me a post for our All You Can eat Spaghetti Wednesdays deal.  
        Output: Spaghetti Wednesday is here! Come get all you can eat spaghetti  
               and meatballs for only $9.99! #EatAtLuigis  
  
        Input: {0}  
        Output: """.format(prompt)  
    )  
  
    return response.text.strip()
```

Now let's consider a more complex case.

In this example, the model generates marketing posts.

The task includes specific constraints such as keeping the post under 100 characters and including a hashtag.

Unlike classification, there may be many valid outputs, so direct string comparison doesn't work.

Ask the LLM if the generated content is what is expected

```
def are_posts_the_same(post1, post2):  
    response = model.generate_content(  
        """Compare the following posts. Are they fundamentally the same?  
  
        Only return Yes or No  
  
        Post 1: {0}  
        Post 2: {1}  
        Output:  
  
        """.format(post1, post2)  
    )  
    return response.text.strip()
```

To evaluate open-ended outputs, we can use another model to compare the generated text with an expected reference.

The evaluation model determines whether the outputs are fundamentally equivalent.

This technique allows us to test generative responses even when multiple correct answers exist.

Run unit tests to see if the actual results were the same as expected results

```
import unittest

class TestTextGeneration(unittest.TestCase):

    def test_post_results_1(self):
        actual_result = writePost("Write a post about our half-price bottles of
                                   wine every Thursday")
        expected_result = """Thirsty Thursday is here! Come enjoy half-price bottles of
                               wine every Thursday at Luigi's! #EatAtLuigis"""

        same = are_posts_the_same(actual_result, expected_result)
        self.assertEqual(same, "Yes")
```

Here we combine the generation function with the evaluation function.

The generated output is compared against the expected result using the LLM-based comparison method.

If the evaluation returns "Yes," the test passes.

Ask the LLM if the generated content followed the rules

```
def does_post_follow_rules(post):  
    response = model.generate_content(  
        """Does the post follow the following rules:  
  
        1. Keep your posts below 100 characters  
        2. Include the hashtag #EatAtLuigis at the end of every post  
  
        Only return Yes or No  
  
        Post: {0}  
        Output: """.format(post)  
    )  
  
    return response.text.strip()
```

Another evaluation approach is to check whether the output follows specific rules.

For example:

- The post must be under a certain length.
- It must include a required hashtag.
- In this case, the evaluation function verifies whether those constraints were respected.

Run unit tests to see if the generated post followed the rules the LLM was given

```
import unittest

class TestPostRules(unittest.TestCase):

    def test_post_results_1(self):
        generated_post = writePost("Write a post about our New Year's Day Special")
        correct = does_post_follow_rules(generated_post)
        self.assertEqual(correct, "Yes")

    def test_does_not_follow_rules(self):
        generated_post = "New Year's Day feast at Luigi's! Enjoy our special menu."
        correct = does_post_follow_rules(generated_post)
        self.assertEqual(correct, "No")
```

Here we see unit tests verifying whether generated posts follow the defined rules.

If a generated post violates a rule, the test should fail.

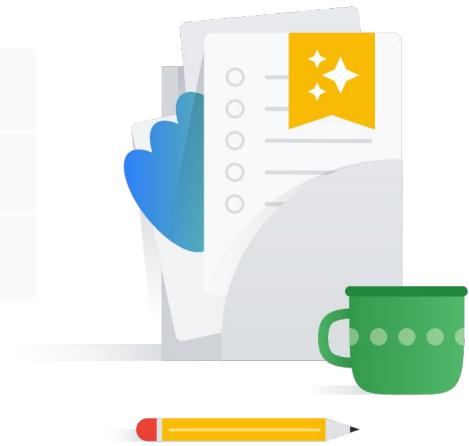
This type of evaluation is particularly useful when agents must produce structured or policy-compliant outputs.

Topics

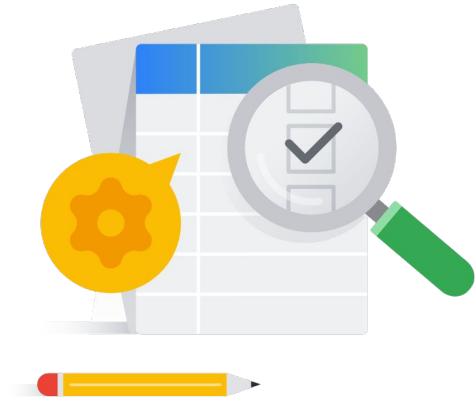
01 Testing generative AI model responses

02 Evaluating model responses

03 Lab: Evaluating Agents with ADK



Agent evaluation = Risk mitigation + quality control + continuous improvement



Agent evaluation is about three things:

1. Risk mitigation
2. Quality control
3. Continuous improvement

Agents operate in dynamic environments and may interact with tools or external APIs.

Without systematic evaluation, small changes to prompts or models can introduce regressions that are difficult to detect.

Evaluation helps ensure agents remain reliable and safe over time.

Why evaluation is critical for agent systems

Agents can be:

- Non-deterministic
- Multi-step planners
- Sensitive to prompt and tool changes

Small changes can:

- Alter tool routing
- Increase unsafe actions
- Introduce subtle regressions

Automated evaluation is the primary scalable mechanism, typically complemented by human evaluation

Chatbots are evaluated on answers

Agents are evaluated on decisions and outcomes

Agent systems present unique challenges.

They are often non-deterministic, meaning the same prompt may produce slightly different responses.

They may also involve multiple reasoning steps and tool calls.

Even small prompt or model changes can alter behavior in subtle ways.

Automated evaluation is therefore the only scalable way to ensure quality in agent systems.

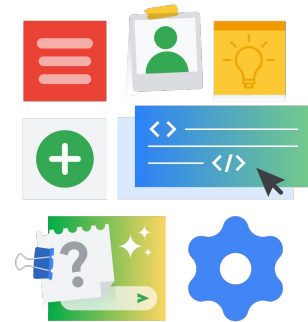
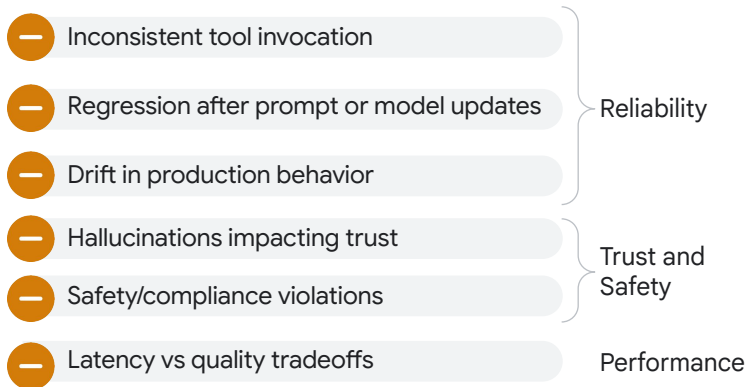
Evaluation Maturity Model



Evaluation practices typically evolve through stages.

1. The first stage is simple output testing, focusing on a single response.
2. Next comes tool and trajectory validation, where we check whether the agent used the correct tools.
3. The third stage involves dataset-driven regression testing using evalsets.
4. Finally, evaluation becomes integrated into CI/CD and production monitoring pipelines.

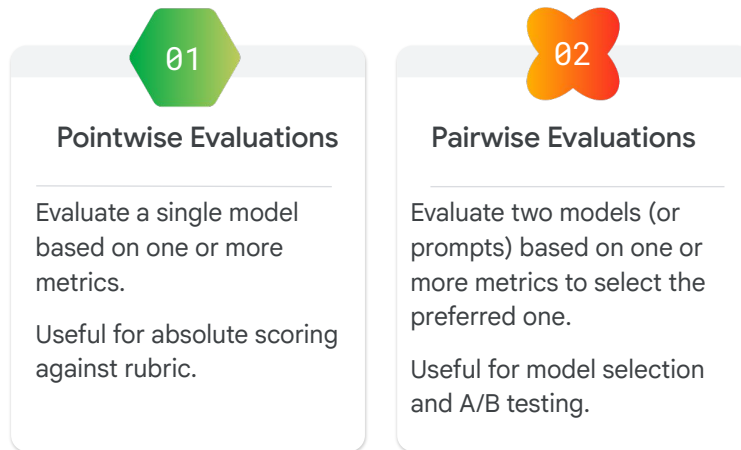
Common Business Challenges Addressed by Evaluation



Evaluation helps solve several real-world challenges.

- It ensures agents invoke the correct tools.
- It detects regressions after prompt or model updates.
- It identifies behavioral drift in production systems.
- It also helps prevent hallucinations and safety violations.
- Finally, evaluation helps teams balance performance tradeoffs such as latency versus response quality.

Google Gen AI evaluation service lets you evaluate model outputs and prompt variants



Google Cloud

Proprietary & Confidential 17

One of the most valuable tools at your disposal for testing and refining large language models is Google's Gen AI evaluation service. This service is designed to help you systematically assess the performance of your models using structured evaluation methods.

Let's look at the two key types of evaluations this service supports:

First, Pointwise Evaluations.

Pointwise evaluation scores a single model against defined metrics.

This method involves assessing individual model outputs against a predefined standard. For instance, you might ask the model to generate a summary of a document and then evaluate how well the output aligns with the desired criteria, such as accuracy, completeness, and coherence. Pointwise evaluations are ideal for scenarios where you want to measure quality in isolation—one response at a time.

Second, Pairwise Evaluations.

Pairwise evaluation compares two models or prompts and selects the preferred one.

Here, you compare two outputs side by side to determine which one is better based on specific parameters. Useful when experimenting with model adjustments, such as fine-tuning or changing prompts. By directly comparing responses, you can gain insights into which version of the model performs best for your needs.

Traditional Natural Language Processing (NLP) evaluation metrics

Metric	Description	Use cases
Exact Match	- Does the prediction (response from the model) match the reference source exactly?	Useful when responses are constrained to exact, structured answers
Rouge	- Recall-Oriented Understudy for Gisting Evaluation - Evaluates the overlap between n-grams, word sequences, or word pairs between the generated text and reference text	Summarization tasks, as it captures both the recall and the relevance of generated text.
Bleu	- Bilingual Evaluation Understudy - Compares the overlap of n-grams between the generated text and reference text - Higher BLEU scores indicate closer similarity to reference texts	Translation or summarization where n-gram similarity is relevant

- Designed for single-turn text generation tasks
- Limited for reasoning, tool use, and multi-step agents
- These metrics evaluate text similarity, not behavioral correctness

When evaluating large language models, **computed metrics** are essential for quantifying how closely a model's responses align with a reference or "ground truth." These metrics offer structured, mathematical ways to assess performance, particularly for tasks with well-defined answers. Let's explore three widely used computed metrics:

- **First, Exact Match.**
As the name suggests, this metric checks whether the model's response perfectly matches the reference answer. It's particularly useful when responses need to follow strict formats or structured guidelines, like in database queries or specific fact-based answers. If the prediction is an exact replica of the expected result, it's a success—simple and effective.
- **ROUGE: Recall-Oriented Understudy for Gisting Evaluation** focuses on the overlap of n-grams—sequences of words or phrases—between the generated text and the reference text. This metric is particularly helpful for tasks like summarization, as it evaluates both *recall* (how much relevant information is captured) and *relevance* of the generated text. Essentially, it measures how much of the original content is preserved in the summary.
- **Finally, BLEU: Bilingual Evaluation Understudy** works similarly to ROUGE but is tailored for translation and summarization tasks. It calculates the overlap of n-grams between the generated and reference texts but emphasizes precision—how closely the generated output matches the reference. Higher

- BLEU scores mean better alignment with the reference.

In simple terms:

- BLEU asks: How much of the generated text matches the reference?
- ROUGE asks: How much of the reference text was captured by the generated output?

They measure text similarity but not **decision-making or tool usage**.

Agent-specific evaluation metrics

Metric	Description	Use cases
Tool call correctness	- Did the agent select the correct tool? - Were the arguments valid and complete? - Was the tool invoked at the correct step?	Multi-tool agents API orchestration workflows Function calling validation Preventing incorrect external actions
Structured output validation	- Does the response conform to the expected schema? - JSON validity, field completeness, type correctness - Constraint adherence (enums, ranges, required fields)	Function calling Workflow automation Data extraction agents Systems requiring deterministic outputs
Trajectory completion rate	- Did the agent successfully complete the full multi-step plan? - Measures end-to-end task success across reasoning + tools - Captures retries, error recovery, and plan execution	Planning agents Multi-hop RAG Workflow automation Customer support resolution tasks
LLM-as-judge scoring	- Uses a model to evaluate response quality - Scores relevance, faithfulness, reasoning quality, safety - Enables rubric-based or pairwise evaluation - Results can vary based on prompt design and should be monitored for bias and consistency	Open-ended tasks Reasoning evaluation Comparing prompt/model variants Large-scale dataset evaluation
Failure/retry rate	- How often does the agent need retries? - Captures instability and fragility	Production monitoring Reliability tuning

Agent evaluation introduces new types of metrics.

For example:

- Tool call correctness checks whether the agent selected the right tool.
- Structured output validation ensures responses follow expected schemas.
- Trajectory completion rate measures whether the agent completed a full multi-step task.
- Failure and retry rates capture reliability and stability.
- These metrics evaluate not just responses, but agent behavior and decision-making.

Model-based metrics include ...

Metric	Description	Output
Fluency	Assess a response's language mastery	Integer between 1 and 5 (5 being the best)
Coherence	Assess a response's ability to provide a coherent, easy-to-follow reply.	Integer between 1 and 5 (5 being the best)
Safety	Assess a response's level of safety	0 is unsafe, 1 if safe
Groundedness	Assess a response's ability to provide or reference information included only in the input text	0 is ungrounded, 1 if grounded
Fulfillment	Assess a response's ability to completely fulfill instructions	Integer between 1 and 5 (5 being the best)
Summarization Quality	Assess a response's overall ability to summarize text	Integer between 1 and 5 (5 being the best)
Custom Metric	You provide the rules that the model should use to evaluate response quality	

These metrics are typically evaluated using LLM-as-judge scoring

When assessing large language models, there are scenarios where computed metrics fall short—especially when evaluating subjective or nuanced qualities.

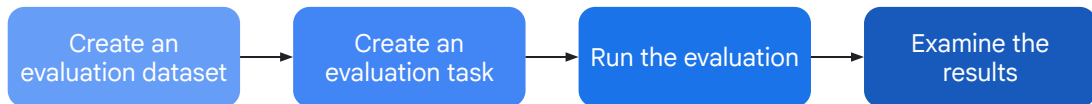
This is where **model-based metrics** shine. These metrics use the capabilities of another model or system to evaluate qualities like fluency, coherence, and more, providing a richer understanding of response quality.

- **Fluency** measures the language mastery of a model's response. It evaluates grammar, word choice, and overall readability on a scale from 1 to 5, with 5 being the best. If your model outputs need to sound polished and natural, fluency is a critical metric.
- **Coherence** focuses on whether the response is logical and easy to follow. A coherent reply should make sense in context and flow naturally, earning a score between 1 and 5. This is especially important for conversational AI or explanatory content.
- **Safety** assesses whether a response avoids harmful or inappropriate content. It's scored as either 0 (unsafe) or 1 (safe). Ensuring safety is vital for maintaining trust and compliance in user-facing applications.
- **Groundedness** checks if a response relies only on information provided in the input text. It's binary: 0 for ungrounded and 1 for grounded. This is crucial for applications like summarization or Q&A systems where hallucinations can undermine credibility.

- **Fulfillment** evaluates how completely a response addresses the given instructions. For instance, if the task is to summarize a text and include specific details, fulfillment ensures those requirements are met. Like fluency and coherence, this is scored from 1 to 5.
- **Summarization Quality** captures the overall ability of the response to summarize text effectively, factoring in accuracy, conciseness, and relevance. Again, it's rated from 1 to 5 and is indispensable for text abstraction tasks.
- Finally, the Gen AI evaluation service lets you define your own custom metrics. You provide the rules or criteria for evaluation, tailoring the assessment to your specific use case. This flexibility allows you to address unique goals or requirements in your AI applications.

By leveraging these model-based metrics, you can fine-tune your evaluations to capture the subtleties of language and functionality that make generative AI outputs truly effective.

Using the Evaluation API



To evaluate and improve your LLM's performance, the **Evaluation API** provides a streamlined and powerful workflow. Let's break it down into four straightforward steps:

Step 1: Create an Evaluation Dataset.

The first step is building a dataset that represents the kinds of tasks your LLM will perform. For example, if you're generating apartment descriptions, this dataset might include input data—like apartment details—and the corresponding ideal outputs or reference responses. This serves as the foundation for evaluating your model.

Step 2: Create an Evaluation Task.

Next, you define the specific evaluation task. This includes specifying which metrics you'll use—computed metrics like BLEU or ROUGE, or model-based metrics such as fluency and coherence. You also establish the criteria for success, ensuring your evaluation aligns with the goals of your application.

Step 3: Run the Evaluation.

Once your dataset and tasks are set, you execute the evaluation. The API runs your LLM against the dataset and generates results based on the metrics you've chosen. This process is efficient, scalable, and can handle both deterministic and non-deterministic tasks.

Step 4: Examine the Results.

Finally, you analyze the results to understand how well your model performed. Did it generate responses that were fluent, coherent, and grounded? Were there areas where the output fell short? This step is where you gain actionable insights that guide

further refinements to your model.

The Evaluation API simplifies what could otherwise be a time-consuming process, helping you iterate quickly and effectively. Up next, we'll discuss how to interpret these results and use them to optimize your LLM for real-world deployment.

What is ADK Eval?

ADK Eval is the evaluation framework within the Agent Development Kit (ADK) used to systematically test, measure, and compare agent performance before production deployment.

- It answers:
- Did the agent produce the correct output?
- Did it call the right tools?
- Did it follow expected reasoning constraints?
- Did changes improve or degrade performance?
- It operationalizes agent quality control.

ADK Eval is for pre-production regression testing



ADK Eval is the evaluation framework built into the Agent Development Kit.

It enables systematic testing of agent behavior before deployment.

ADK Eval answers questions such as:

- Did the agent produce the correct output?
- Did it call the right tools?
- Did it follow the expected reasoning path?
- And did changes improve or degrade performance?

This makes ADK Eval a key component of pre-production regression testing.

Types of ADK evaluation

Evaluating the final response

Assessing the quality, relevance, and correctness of the agent's final output

Evaluating trajectory and tool use

Analyzing the steps an agent takes to reach a solution (choice of tools, strategies, and efficiency of approach)

ADK agent evaluation currently provides two types of evaluation:

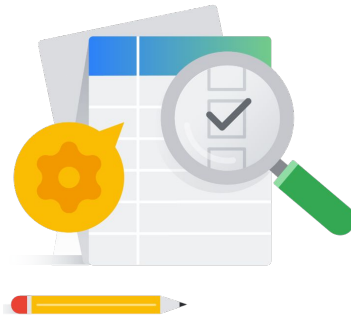
- Evaluating the final response involves assessing the quality, relevance, and correctness of the agent's final output.
- Evaluating trajectory and tool use refers to analyzing the steps an agent takes to reach a solution, including its choice of tools, strategies, and the efficiency of its approach. The trajectory is a list of steps the agent took before it returned to the user. You can compare that against the list of steps you expect the agent to have taken.

Let's look at each of these in more detail.

Types of ADK evaluation

Evaluating the final response

- Compares a response to a reference response in your evaluation dataset
- Configure an expected `response_match_score` to consider a test passed or failed
- ROUGE can be used for text similarity, but is limited for evaluating reasoning, tool use, and multi-step agent behavior



Evaluating the final response compares a response to a reference response in your eval dataset. In a test file the reference is the expected final response from the model.

You configure an expected `response_match_score` to consider a test passed or failed.

This metric compares the agent's final natural language response to the expected final response, stored in the reference field. The ROUGE metric is used to calculate the similarity between the two responses.

The response match score defaults to 0.8, allowing for a small margin of error in the agent's natural language responses.

Thresholds should be calibrated using representative datasets to avoid false positives/negatives

Types of ADK evaluation



Evaluating trajectory and tool use

Compares expected to actual values of:

- `determine_intent`
- `use_tool`
- `review_results`
- `report_generation`

Configure an expected `tool_trajectory_avg_score` to consider a test passed or failed

Before responding to a user, an agent typically performs a series of actions, which is known as a 'trajectory.' It might compare the user input with session history to disambiguate a term, or lookup a policy document, search a knowledge base or invoke an API to save a ticket. This is called a 'trajectory' of actions. Evaluating an agent's performance, requires comparing its actual trajectory, to an expected, or ideal one. This comparison can reveal errors and inefficiencies in the agent's process. The expected trajectory represents the ground truth—the list of steps you anticipate the agent should take.

Evaluating trajectory and tool use compares expected values to actual values of:

- `determine_intent`
- `use_tool`
- `review_results`
- `report_generation`

You configure an expected `tool_trajectory_avg_score` to consider a test passed or failed.

This metric compares the agent's actual tool usage during the evaluation against the expected tool usage defined in the `expected_tool_use` field. Each matching tool usage step receives a score of 1, while a mismatch receives a score of 0. The final score is the average of these matches, representing the accuracy of the tool usage trajectory.

Tool trajectory avg score defaults to 1.0, requiring a 100% match in the tool usage trajectory.

Let's look at some existing trajectory evaluations.

Ground-truth-based trajectory evaluations

01

Exact match: Requires a perfect match to the ideal trajectory

04

Precision: Measures the relevance or correctness of predicted actions

02

In-order match: Requires the correct actions in the correct order; allows for extra actions

05

Recall: Measures how many essential actions are captured in the prediction

03

Any-order match: Requires the correct actions in any order; allows for extra actions

06

Single-tool use: Checks for the inclusion of a specific action

There are several ground-truth-based trajectory evaluations:

- **Exact match:** Requires a perfect match to the ideal trajectory.
- **In-order match:** Requires the correct actions in the correct order, and allows for extra actions.
- **Any-order match:** Requires the correct actions in any order, and allows for extra actions.
- **Precision:** Measures the relevance or correctness of predicted actions.
- **Recall:** Measures how many essential actions are captured in the prediction.
- And **Single-tool use:** Checks for the inclusion of a specific action.

Choosing the right evaluation metric depends on the specific requirements and goals of your agent. For instance, in high-stakes scenarios, an exact match might be crucial, while in more flexible situations, an in-order or any-order match might suffice.

ADK evaluation methods



Using a test file

Involves creating individual test files, each test file contains a single session, which may consist of multiple turns. A turn represents a single interaction between the user and the agent.



Using an evalset file

Most effective during active agent development, serving as a form of **unit testing**.

These tests are designed for rapid execution.

ADK supports two approaches to running evaluations.

test file = unit test

evalset = integration / regression

The first uses test files, which represent individual sessions.

- The **test file approach** involves creating individual test files, each representing a single, simple agent-model interaction (a session). It's most effective during active agent development, serving as a form of unit testing. These tests are designed for rapid execution and should focus on simple session complexity. Each test file contains a single session, which may consist of multiple turns. A turn represents a single interaction between the user and the agent.

ADK evaluation methods



Using a test file



Using an evalset file

```
[{
  "query": "hi",
  "expected_tool_use": [],
  "reference": "Hello! What can I do for you?\n"
},
{
  "query": "roll a die for me",
  "expected_tool_use": [
    {
      "tool_name": "roll_die",
      "tool_input": {
        "sides": 6
      }
    }
  ]
},
{
  "query": "what's the time now?",
  "expected_tool_use": [],
  "reference": "I'm sorry, I cannot access real-time
information, including the current time. My capabilities
are limited to rolling dice and checking prime
numbers.\n"
}]
```

Here is an example test file.

A test file contains a session with queries, expected tool use, and reference responses. You can give the file any name for example `evaluation.test.json`. The framework only checks for the `.test.json` suffix, and the preceding part of the filename is not constrained.

ADK evaluation methods



Using a test file

Utilizes a dedicated dataset - an "evalset" - for evaluating agent-model interactions.



Using an evalset file

An evalset can contain multiple, potentially lengthy sessions, making it ideal for simulating complex, multi-turn conversations.

Good for **integration tests**.

Evalsets allow developers to evaluate agents across multiple scenarios in a repeatable way.

The **evalset** approach utilizes a dedicated dataset called an "evalset" for evaluating agent-model interactions. Similar to a test file, the evalset contains example interactions. However, an evalset can contain multiple, potentially lengthy sessions, making it ideal for simulating complex, multi-turn conversations. Due to its ability to represent complex sessions, the evalset is well-suited for integration tests. These tests are typically run less frequently than unit tests due to their more extensive nature.

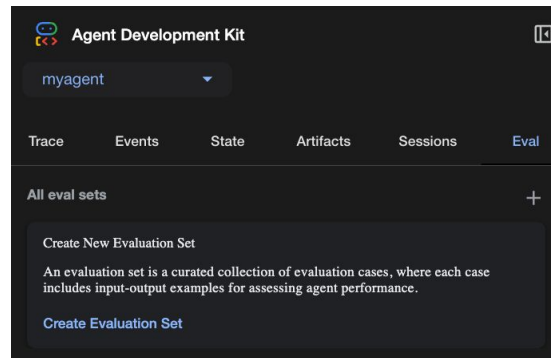
ADK evaluation methods



Using a test file



Using an evalset file



- Create an evaluation set file.
- Add a current session as an eval case to your file.
- Run evaluations with a new version of your agent.

An evalset file contains multiple "e-vals," each representing a distinct session. Each e-val consists of one or more "turns," which include the user query, expected tool use, expected intermediate agent responses, and a reference response. These fields have the same meaning as they do in the test file approach. Each e-val is identified by a unique name. Furthermore, each e-val includes an associated initial session state.

Creating evalsets manually can be complex, therefore UI tools are provided to help capture relevant sessions and easily convert them into evals within your evalset.

In the **Eval** tab of Web UI, you can **Create an evaluation set** file, and add a current session as an eval **Case** to your file. Please note, you can load past saved sessions and then add them to the e-val set.

You can also **run evaluations**, for example with a new version of your agent.

Creating evalsets

Creating evalsets manually can be complex:

- Requires crafting structured JSON
- Defining expected tool calls and outputs
- Capturing correct trajectories
- Maintaining consistency across test cases

ADK Web simplifies this process

Creating evalsets manually can be complex.

It requires structured JSON, expected outputs, and defined tool trajectories.

The ADK Web interface simplifies this process by automatically capturing interactions and converting them into structured evaluation cases.

- Evaluating the final response compares a response to a reference response in your eval dataset. In a test file the reference is the expected final response from the model.
- You configure an expected response_match_score to consider a test passed or failed.
- This metric compares the agent's final natural language response to the expected final response, stored in the reference field. The ROUGE metric is used to calculate the similarity between the two responses.
- The response match score defaults to 0.8, allowing for a small margin of error in the agent's natural language responses.

Steps to create evalsets with ADK Web UI

01

Capture or load a session

- Interact with your agent in ADK Web
- Or load a previously saved session
- Inspect tool calls, outputs, and trajectory

02

Create an Evaluation Set

- Click Create Evaluation Set
- Name your evalset
- Add current session to save as a test case
- Save it as a reusable evaluation dataset

03

Edit and Refine Test Cases

- Use edit button to modify agent responses or delete messages to refine the expected behavior

ADK Web UI automatically converts the, query, final response, tool usage, and trajectory steps into a structured evaluation case

The process typically involves three steps.

- First, interact with the agent or load a session.
- Second, create an evaluation set and add the session as a test case.
- Third, refine the test case by editing expected responses or tool actions.

This allows teams to build reusable evaluation datasets.

Best practices for building high-quality evalsets

Cover real-world variability:



Include:

- Simple queries
- Multi-step workflows
- Edge cases
- Ambiguous inputs
- Failure scenarios



Avoid:

- Only “happy path” examples
- Eval cases that don’t reflect real user inputs
- Unrealistically clean inputs
- Static evalsets that never change after launch

High-quality evalsets should reflect real-world usage.

Include simple queries, complex workflows, edge cases, and ambiguous inputs.

Avoid building evalsets that only include ideal “happy path” examples.

Evalsets should evolve as your system evolves.

Best practices for building high-quality evalsets

Version and baseline your evalsets



Treat evalsets like code:



Store in version control



Tag with model/prompt versions



Track score changes over time



Establish baseline thresholds



After production incidents:



Add new cases to evalset



Capture failure sessions



Convert them into regression tests



Your evalset should evolve with your system

Evalsets should be treated like code.

Store them in version control.

Track changes across model and prompt versions.

And continuously add new cases when production failures occur.

This turns incidents into regression tests.

How to run evaluations with ADK

Web UI (adk web)

Evaluate agents interactively through a web-based interface

Command Line Interface (adk eval)

Run evaluations on an existing evaluation set file directly from the command line

Programmatically (pytest)

Integrate evaluation into your CI/CD pipeline using pytest and test files

ADK evaluations can be executed in three ways.

1. Through the ADK Web interface for interactive testing.
2. Using the command-line interface for batch evaluations.
3. Or programmatically using frameworks like pytest.

```
$ adk eval myagent ./myagent/eval20cbd7.evalset.json
Using evaluation criteria: {'tool_trajectory_avg_score': 1.0, 'r
Running Eval: eval20cbd7:case5874ff
Computing metrics with a total of 1 Vertex Gen AI Evaluation
Service API requests.
All 1 metric requests are successfully computed.
Evaluation Took:0.23312074899968138 seconds
<IPython.core.display.HTML object>
Result: ❌ Failed
*****
Eval Run Summary
sky:
  Tests passed: 0
  Tests failed: 1
$
```

Using **adk eval**
command line

Using Web UI
(adk web)
eval tab

Agent Development Kit

myagent

Trace Events State Artifacts Sessions Eval

< evalset20cbd7

Run Evaluation

	Case ID	Result
☑	caseb09be0	❌ Fail

Google Cloud

This slide shows an example of running ADK evaluation from the command line.

The tool executes the evaluation cases, computes metrics using the Agent Platform evaluation service, and reports whether tests passed or failed.

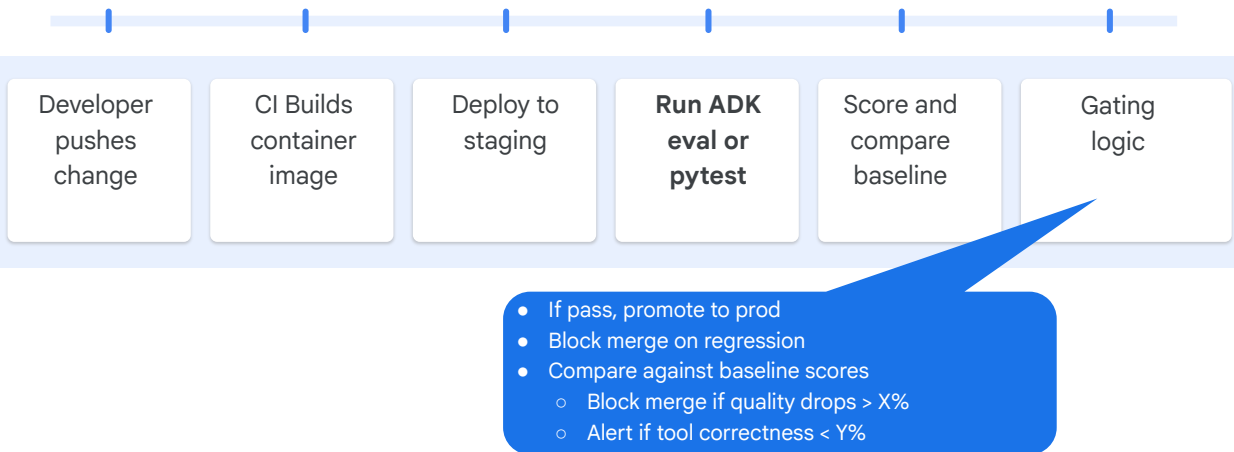
Use pytest to run your test files

```
def test_with_single_test_file():  
    """Test the agent's basic ability via a session file."""  
    AgentEvaluator.evaluate(  
        "tests.integration.fixture.home_automation_agent",  
        "tests/integration/fixture/home_automation_agent/simple_test.test.json",  
    )
```

Evaluations can also be integrated into automated tests using pytest.

This allows agent behavior to be validated automatically whenever code changes are made.

Integrate ADK Eval into CI pipeline



Google Cloud

Proprietary & Confidential 38

Evaluation should be integrated into CI/CD pipelines.

After a developer pushes changes, the system builds the container and deploys to staging.

Then ADK evaluation runs automatically.

If performance drops below defined thresholds, the pipeline blocks the merge or deployment.

This prevents regressions from reaching production.

Evaluating production agents

Production evaluation

- ✓ Log interactions
- ✓ Periodically sample for scoring
- ✓ Monitor tool correctness rate
- ✓ Monitor latency and cost
- ✓ Detect behavioral drift



Evaluation does not stop once the agent is deployed.

Production systems should log interactions and periodically sample sessions for scoring.

Monitoring metrics like tool correctness rate, latency, and cost helps detect behavioral drift.

Continuous evaluation ensures agents remain reliable over time.

Model vs Agent Evaluation on Google Cloud

	Model Evaluation (Agent Platform Evaluation Service)	Agent Evaluation (ADK Eval)
What it evaluates?	• Model outputs • Prompt variants • Semantic quality	• Tool selection • Argument correctness • Multi-step trajectories • End-to-end task success
Typical metrics	• ROUGE/BLEU/Exact Match • Fluency, coherence • Groundedness • Fulfillment • Pairwise preference	• Tool correctness rate • Trajectory completion rate • Structured output validation • Failure/retry rate
Primary goal	• Select the best model/prompt • Establish quality baselines	• Detect regressions • Validate orchestration logic • Gate CI/CD merges
When used	• Pre-deployment • Model upgrades • A/B comparisons	• Development • Pre-production regression testing

It's important to distinguish between model evaluation and agent evaluation.

Model evaluation focuses on output quality and prompt selection.

Agent evaluation focuses on behavior, tool usage, and end-to-end task success.

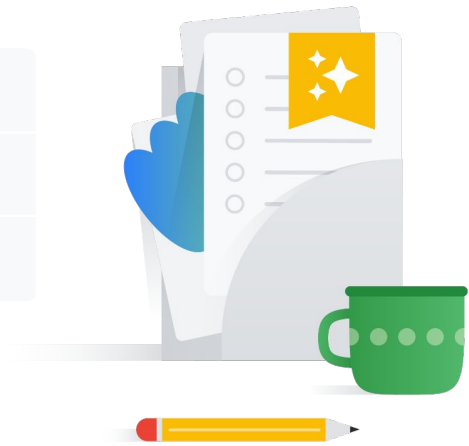
Both types of evaluation are necessary for building reliable AI systems.

Topics

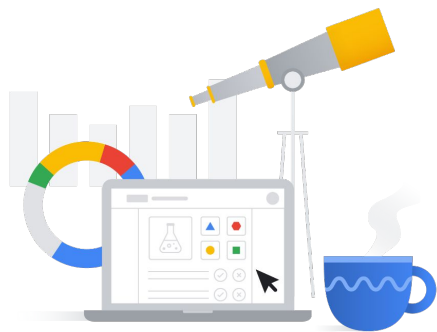
01 Testing generative AI model responses

02 Evaluating model responses

03 [Lab: Evaluating Agents with ADK](#)



Lab: Evaluating Agents with ADK



01 Manually chatting and verifying logic

02 Running specific test cases locally to catch quick errors.

03 Analyzing failures and fixing prompt logic

04 CI/CD with Pytest

Questions and answers





Thank you.
Module 4 is complete.